

Phase 3 – System Design Review

Deliverables Report

Project Title:

Smart Vehicle Accident Detection and Emergency Alert System

Objective:

To design an embedded system that automatically detects a vehicle collision using motion/impact sensors, determines the vehicle's real-time GPS location, and instantly alerts pre-configured emergency contacts and/or a control room via SMS/call, thereby reducing emergency response time and potentially saving lives.

Deliverable Sections Covered in this Document:

- System Architecture (Overall Architecture, Data Flow, Functional Block Diagram, HW-SW Partitioning)
- Hardware Design (Block Diagram, Schematic, Component Selection, BOM, PCB Layout)
- Software Design (Flowcharts, Algorithms, Communication Protocols, Database Structure)
- Simulation (Tools Used, Initial Results)

1. System Architecture

1.1 Overall Solution Architecture

The system follows a layered embedded-IoT architecture consisting of a sensing layer, a processing layer, a communication layer, and an alerting layer, all supported by a regulated power supply unit. Motion data from the MPU6050 accelerometer/gyroscope, along with the manual panic button state, are continuously fed to the ESP32 microcontroller. The ESP32 runs a threshold-based crash-detection algorithm; when a collision is confirmed, it retrieves the current GPS coordinates from the NEO-6M module and uses its built-in Wi-Fi to send the location link, timestamp, and vehicle/rider ID to a cloud service (e.g. Firebase or ThingSpeak), which in turn notifies pre-configured emergency contacts by SMS/email/app notification (via a service such as IFTTT or Twilio). A local buzzer and LED provide an immediate audible/visual indication independent of network availability.

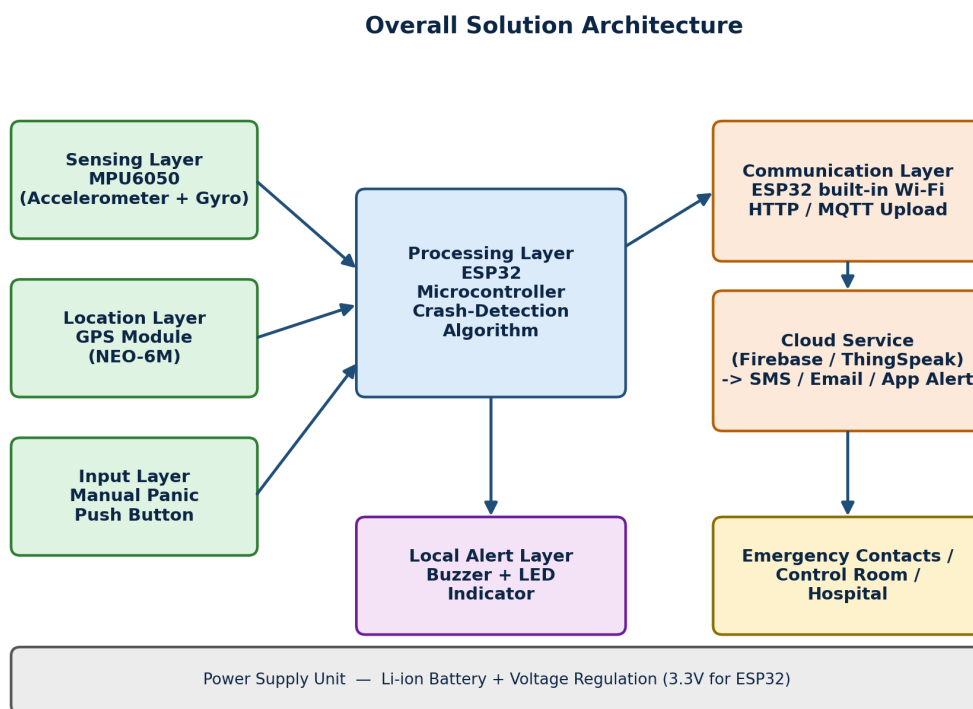


Figure 1.1 – Overall Solution Architecture of the Smart Vehicle Accident Detection and Emergency System

1.2 Data Flow Diagram

The Data Flow Diagram (DFD) below traces how raw sensor data is transformed into an outbound emergency alert. Process 1 continuously acquires and compares sensor readings against a calibrated threshold. On a positive detection, Process 2 composes the alert payload (GPS coordinates, timestamp, and device ID) and uploads it over Wi-Fi, while Process 3 concurrently triggers the local audio-visual alarm. The uploaded data reaches a cloud service, which forwards the alert to the stored emergency contacts.

Data Flow Diagram

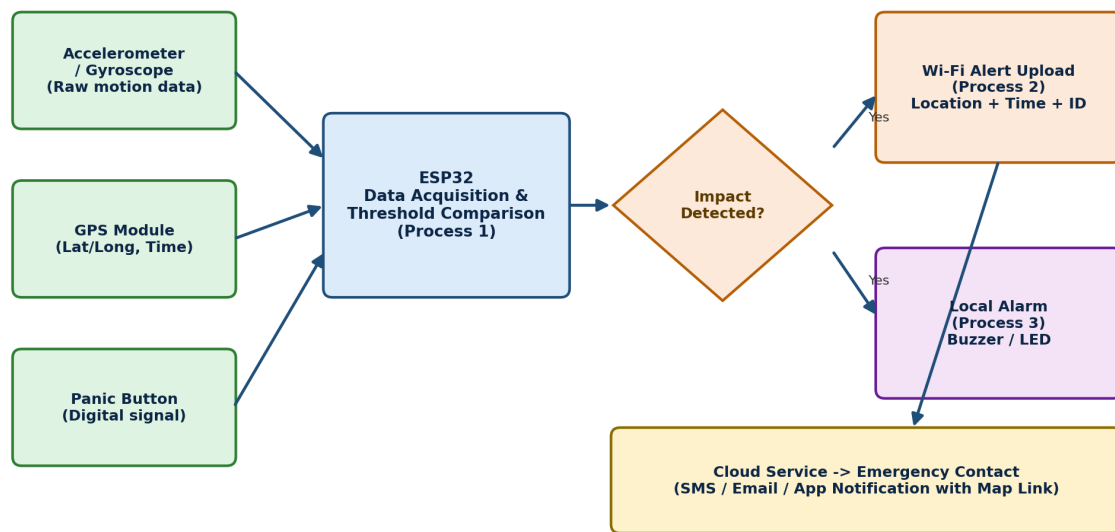


Figure 1.2 – Data Flow Diagram showing sensor-to-alert data transformation

1.3 Functional Block Diagram

Functionally, the system can be grouped into four cooperating blocks: (1) the **Sensing & Input Block** (MPU6050 accelerometer/gyroscope, panic button), (2) the **Processing Block** (ESP32 running the detection algorithm and system logic), (3) the **Communication Block** (GPS for positioning, and the ESP32's built-in Wi-Fi for uploading alerts to a cloud service), and (4) the **Output/Alerting Block** (buzzer, LED, and the emergency contact/control room endpoint reached via the cloud). This grouping is the same as depicted in the architecture diagram of Section 1.1, and is used consistently through the hardware and software designs that follow.

1.4 Hardware-Software Partitioning

The partitioning below defines which responsibilities are implemented purely in hardware versus in the embedded firmware, to clarify design boundaries before detailed design.

Function	Hardware Responsibility	Software Responsibility
Motion / Impact Sensing	MPU6050 accelerometer + gyroscope provides raw motion signals over I2C	Read raw values over I2C, apply filtering, compute resultant acceleration
Crash Detection	Sensor electrical interfacing only	Threshold comparison algorithm, false-positive rejection logic (e.g. debounce, multi-axis check)
Location Tracking	NEO-6M GPS module acquires satellite fix	Parse NMEA sentences, extract latitude/longitude/time via UART
Alert Transmission	ESP32 built-in Wi-Fi radio performs RF transmission	Wi-Fi connection handling, HTTP/MQTT request to cloud service with composed alert payload
Local Indication	Buzzer and LED as physical output devices	PWM/digital signal generation, alert pattern timing
User Override	Push button as physical input	Debounce logic, 15-second cancellation window state machine
Power Management	Li-ion battery, protection circuit, regulator provide stable voltage	Low-power / sleep mode scheduling in firmware (optional)

2. Hardware Design

2.1 Final Block Diagram

The final hardware block diagram elaborates the architecture into concrete components and their electrical interfaces (I2C, UART, digital I/O) to the microcontroller, forming the basis for the schematic and BOM that follow.

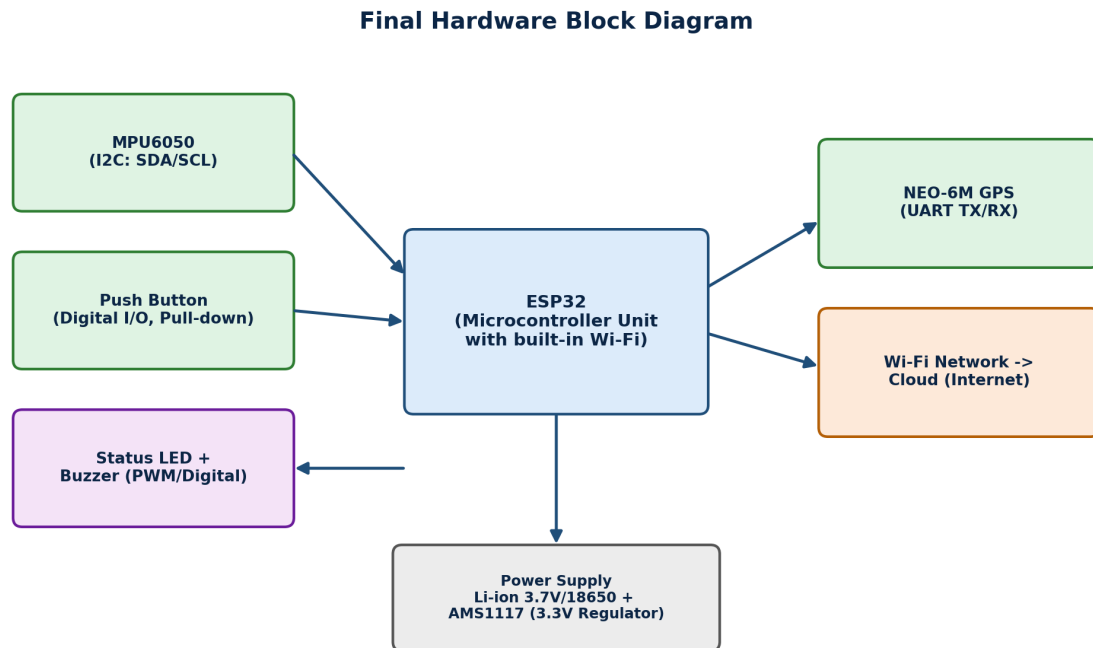


Figure 2.1 – Final Hardware Block Diagram with interface types

2.2 Circuit Schematic

A simplified schematic is shown below indicating ESP32 pin assignments. The MPU6050 communicates over I2C (GPIO21/SDA, GPIO22/SCL); the GPS module uses a dedicated hardware UART pair (GPIO16/RX2, GPIO17/TX2); the push button uses a digital input pin (internal pull-down); and the buzzer/LED are driven from digital output pins, the buzzer through a small NPN transistor driver and the LED through a current-limiting resistor. All modules share a common 3.3V rail and ground, regulated from the Li-ion battery through an AMS1117 regulator.

Simplified Circuit Schematic

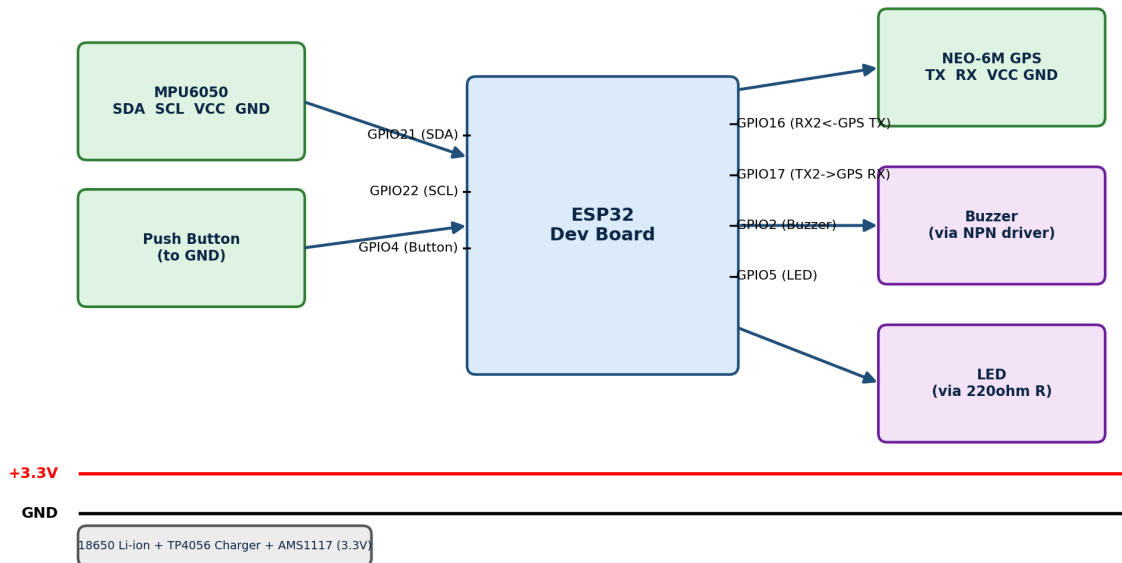


Figure 2.2 – Simplified Circuit Schematic and Pin Mapping

2.3 Component Selection

Component	Selected Part	Justification
Microcontroller	ESP32 Dev Board	Dual-core processor, ample GPIO/UART/I2C peripherals, and native Wi-Fi for direct cloud alert delivery without a separate GSM module
Motion Sensor	MPU6050 (3-axis accel + 3-axis gyro)	Low cost, I2C interface, widely supported libraries, adequate sensitivity for crash detection
GPS Module	NEO-6M	Reliable cold-start accuracy, UART NMEA output, low power, common in embedded trackers
Alert Output	Piezo Buzzer + High-brightness LED	Immediate low-power local indication independent of network availability
Input	Tactile Push Button	Manual override / false-alarm cancellation
Power Source	3.7V Li-ion 18650 cell + TP4056 charger	Rechargeable, compact, suited to in-vehicle installation
Voltage Regulation	AMS1117 (3.3V LDO Regulator)	Stable regulated 3.3V supply matched to ESP32 and logic-level module requirements

2.4 Bill of Materials (BOM)

#	Component	Qty	Unit Price (approx.)	Total (approx.)
1	ESP32 Dev Board	1	Rs. 550	Rs. 550
2	MPU6050 Accelerometer + Gyroscope Module	1	Rs. 120	Rs. 120
3	NEO-6M GPS Module	1	Rs. 550	Rs. 550
4	Piezo Buzzer	1	Rs. 15	Rs. 15
5	5mm LED (Red) + 220ohm resistor	2	Rs. 5	Rs. 10
6	Tactile Push Button	1	Rs. 5	Rs. 5
7	18650 Li-ion Cell (3.7V, 2200mAh)	1	Rs. 180	Rs. 180
8	TP4056 Charging Module	1	Rs. 30	Rs. 30
9	AMS1117 3.3V Regulator Module	1	Rs. 25	Rs. 25
10	PCB / Perfboard + Connectors + Wires	1 set	Rs. 150	Rs. 150
11	Enclosure Box	1	Rs. 100	Rs. 100

Estimated total prototype cost: approximately Rs. 1,735 (costs are indicative and vary by vendor/region; a Wi-Fi network with internet access is required at the vehicle location for the ESP32 to reach the cloud service — no SIM card is needed since there is no GSM module in this configuration).

2.5 PCB Layout (Indicative)

At the prototyping stage, modules are typically interconnected on a perfboard or via jumper wires on a breadboard rather than a fabricated PCB. An indicative layout is provided below for reference; a compact custom PCB can be developed in a later phase using KiCad or Eagle once the design is validated, incorporating the same module placement and trace routing principles.

Indicative PCB / Perfboard Layout (Prototype Stage)

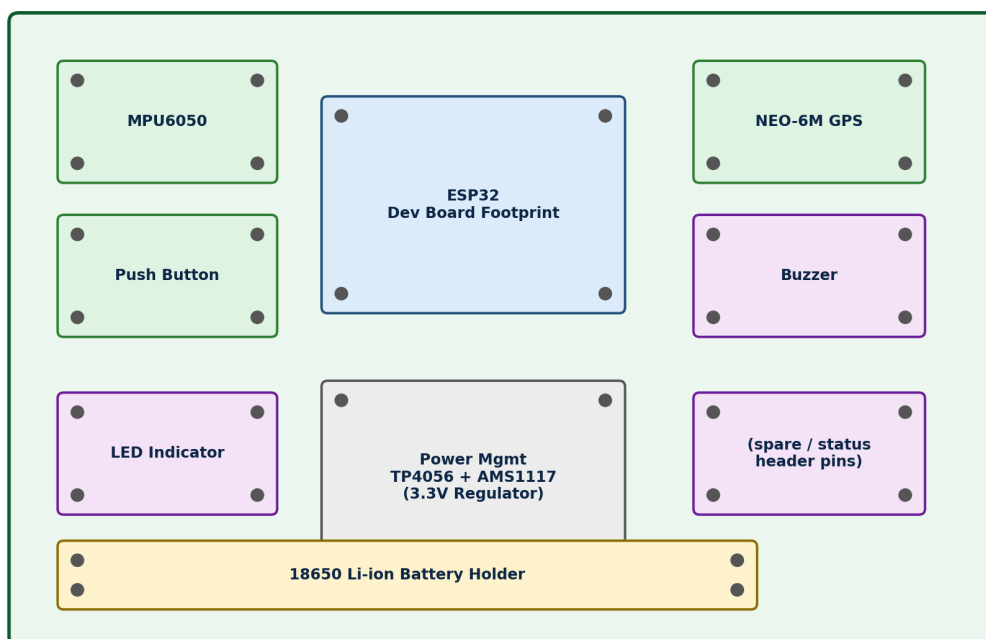


Figure 2.5 – Indicative PCB / Perfboard Layout for the Prototype

3. Software Design

3.1 Flowcharts

The firmware operates as a continuous monitoring loop. After initializing all peripherals, the system repeatedly samples the accelerometer/gyroscope, compares the computed change against a calibrated threshold, and — on detection (or manual panic-button press) — starts a short cancellation window before dispatching the emergency alert. This allows the user to cancel false triggers (e.g. hitting a pothole) before an alert is sent.

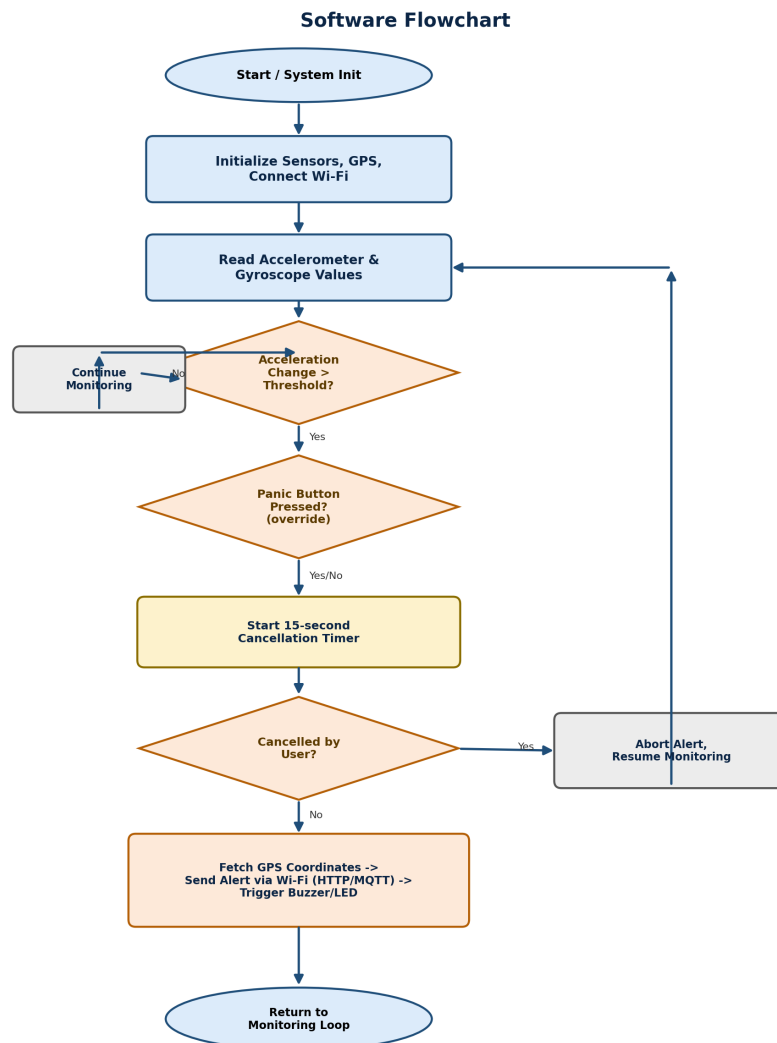


Figure 3.1 – Main Firmware Flowchart

3.2 Algorithms

Crash Detection Algorithm (threshold-based): The core detection logic computes the resultant acceleration magnitude from the 3-axis accelerometer and compares it against a calibrated crash threshold (commonly 2.5g – 4g depending on vehicle type and sensor mounting). Pseudocode:

```
1.  INITIALIZE MPU6050, GPS, I/O pins, connect ESP32 to Wi-Fi
2.  LOOP forever:
3.      read ax, ay, az from MPU6050
4.      compute resultant = sqrt(ax^2 + ay^2 + az^2)
5.      IF resultant > CRASH_THRESHOLD OR panic_button == PRESSED:
6.          start cancel_timer (15 seconds)
7.          WHILE cancel_timer running:
8.              IF cancel_button pressed:
9.                  abort alert; BREAK to step 2
10.             fetch GPS latitude, longitude
11.             compose_payload = {device_id, lat, lon, timestamp, g_force}
12.             send_HTTP_POST(cloud_endpoint, compose_payload)
13.             cloud service forwards SMS / email / app notification to emergency contacts
14.             trigger buzzer + LED pattern
15.             log event (locally / to cloud)
16.      ELSE:
17.          continue monitoring
```

A sustained multi-sample check (requiring the threshold to be exceeded across consecutive readings, combined with gyroscope orientation change) reduces false positives from ordinary road bumps, hard braking, or potholes. The cancellation window ensures the driver/rider can call off an accidental trigger.

3.3 Communication Protocols

Interface	Protocol	Purpose
ESP32 <-> MPU6050	I2C (400 kHz)	Read acceleration and gyroscope registers
ESP32 <-> NEO-6M GPS	UART (Hardware Serial2), 9600 baud, NMEA-0183 sentences	Receive GPRMC/GPGGA sentences for lat/long/time
ESP32 -> Push Button	Digital GPIO (interrupt-capable)	Event-driven manual override / cancellation input
ESP32 -> Buzzer/LED	Digital / PWM GPIO	Local audio-visual alert pattern
ESP32 -> Cloud Service	Wi-Fi (802.11 b/g/n), HTTPS/MQTT over TCP-IP	Push accident event and GPS log to Firebase/ThingSpeak, which triggers SMS/email/app alerts

3.4 Database Structure (Cloud Logging)

Each accident event is uploaded via Wi-Fi and logged in the cloud service as a single record with the structure below, enabling a control room to review historical incidents and trigger the appropriate SMS/email/app notification workflow.

Field	Type	Description
event_id	INTEGER (Primary Key)	Unique identifier for each accident event
device_id	VARCHAR(20)	Unique ID of the installed unit / vehicle
timestamp	DATETIME	Date and time the event was detected
latitude	FLOAT	GPS latitude at time of impact

Field	Type	Description
longitude	FLOAT	GPS longitude at time of impact
impact_g_force	FLOAT	Peak resultant acceleration recorded (in g)
alert_status	VARCHAR(15)	Sent / Cancelled / Failed
contact_notified	VARCHAR(100)	Emergency contact number(s) notified

4. Simulation

4.1 Simulation Tools

Before hardware assembly, the design is validated in simulation to verify logic and approximate sensor behaviour:

- **Wokwi** – used for simulating the ESP32 firmware together with a virtual MPU6050, a virtual GPS module, and a simulated Wi-Fi connection, allowing the crash-detection algorithm and the HTTP alert request to be tested without physical hardware.
- **Proteus** – used for simulating the overall circuit (ESP32 + peripheral modules) and verifying pin-level electrical connections prior to assembly.
- **Tinkercad Circuits** – used for quick breadboard-style logic verification of button/buzzer/LED behaviour and basic sensor interfacing.
- **MATLAB** (optional) – used to analyze sample accelerometer waveform data offline and help tune the crash-detection g-force threshold before deploying it to firmware.

4.2 Initial Results

A simulated impact test was run by injecting a short high-magnitude acceleration pulse into the virtual accelerometer input while the system was in normal monitoring state. The plot below shows the resultant acceleration crossing the calibrated 3.5g threshold at $t \approx 6.0\text{s}$, which correctly triggered the alert routine (GPS fetch + simulated Wi-Fi HTTP alert request + buzzer activation) in the firmware log.

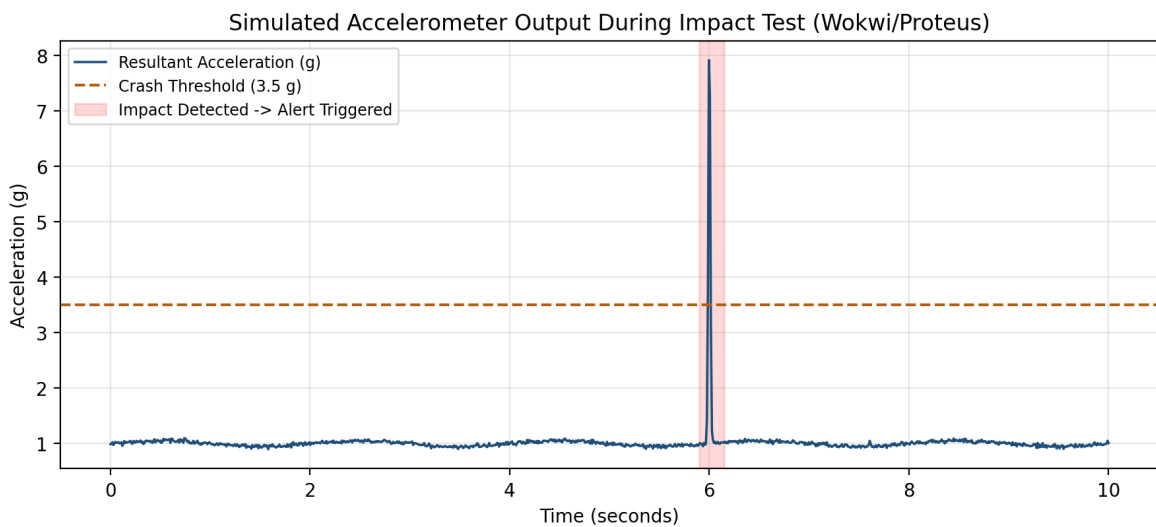


Figure 4.2 – Simulated Accelerometer Output During an Impact Test

Summary of Initial Observations

Test Parameter	Observation
Idle-state acceleration reading	~1.0g (steady, with expected minor sensor noise)
Crash threshold set for test	3.5g resultant acceleration
Detection response time (simulated)	< 200 ms from threshold crossing to alert trigger
False trigger during idle/road-noise phase	None observed in the simulated run
Cancellation window behaviour	15-second countdown correctly aborts alert when cancel input is simulated

Test Parameter	Observation
GPS parsing	NMEA sentence correctly parsed to latitude/longitude in Wokwi virtual GPS
Wi-Fi alert dispatch (HTTP POST)	Request to simulated cloud endpoint completed successfully in serial log

These initial results confirm the core detection-to-alert pipeline behaves as designed in simulation. Physical prototyping and field testing (e.g. controlled drop/impact tests on the assembled hardware, live GPS fix acquisition outdoors, and confirming the ESP32 connects reliably and delivers alerts over a real Wi-Fi network) are the next steps planned for the following project phase.